

DESIGN DOCUMENT

PROGRAM OVERVIEW

This project generates new Sudoku puzzles by randomly shuffling an existing solution 6 different ways!

1. *Swapping digits*
2. *Transposing the board*
3. *Shuffling rows of boxes*
4. *Shuffling columns of boxes*
5. *Shuffling rows within boxes*
6. *Shuffling columns within boxes*

All six of these operations preserve the validity of the original board!

DATA STRUCTURES

I used a **9x9 matrix** (in the form of a **2D integer array**) to store the board. The **rows** and **columns** correspond to the rows and columns of the Sudoku board (ignoring box boundaries). Each box was defined by a **boxRow** and a **boxColumn**. The **origin** of a box is the top-left corner of the box. The **rowInBox** and **columnInBox** format treats each box as its own matrix. This format allows for access to specific positions within specific boxes.

ALGORITHM DESIGN

The algorithm creates a new puzzle in **6 steps** starting from a **pre-existing solution**. These 6 operations **preserve the validity** of the board while altering the digits.

1. Swapping digits

Each digit (1-9) is mapped to another random digit (through 9 random swaps). This preserves the set's length and contents. Then this map is applied to the board, replacing each digit with its counterpart.

2. Transposing the board *50% chance of transposing

Transposing is the process of swapping the rows/columns of a matrix (essentially rotating it). This has a 50% chance of occurring to add to the randomness of the result.

3. Shuffling rows of boxes

The board can be divided into 9 boxes. 3 rows of 3 boxes. These rows are then randomly swapped with each other 3 times over.

4. Shuffling columns of boxes

The board can be divided into 9 boxes. 3 columns of 3 boxes. These columns are then randomly swapped with each other 3 times over.

5. Shuffling rows within boxes

Within each row of boxes there are 3 full rows of elements (spanning the board). These rows are randomly swapped with each other 3 times over.

6. Shuffling columns within boxes

Within each column of boxes there are 3 full columns of elements (spanning the board). These columns are randomly swapped with each other 3 times over.

METHODS

`generate ( )`

Generates a brand new board and stores it in the buffer.

`isValid (int[ ][ ] board)`

Determines if a board is a valid sudoku board.

`isValidGroup (int[ ] group)`

Determines whether or not the array is a valid sudoku group.

`swapRows (int row1, int row2)`

Swaps two rows in the board buffer.

swapColumns (int column1, int column2)

Swaps two columns in the board buffer.

swapBoxRows (int boxRow1, int boxRow2)

Swaps two rows of boxes.

swapBoxColumns (int boxColumn1, int boxColumn2)

Swaps two columns of boxes.

shuffleDigits ()

Swaps the digits within the board randomly.

shuffleRowsWithinBoxRows ()

Shuffles the rows within each horizontally aligned box.

shuffleColumnsWithinBoxColumns ()

Shuffles the columns within each vertically aligned box.

shuffleBoxRows ()

Shuffles box rows.

shuffleBoxColumns ()

Shuffles box columns.

transpose ()

Transposes the board (swaps rows and columns).

board ()

A new board is generated on each call of the 'generate' method. This board is then stored in a buffer and accessed with this method.

copyInto (int[][] board, int[][] buffer)

Copies a board into another board's buffer.

printBoard ()

A new board is generated on each call of the 'generate' method. This board is then stored in a buffer and printed to the console with this method.

TESTING PLAN

Case 1: **MY EYES**

I looked at the board. It seems legit! 👍

Case 2: **CODE**

I coded a validator method. Ran it. Seems legit as well.

Case 3: **THE BIG GPT**

Hey *ChatGPT*, is this a valid Sudoku? *Yes, this is indeed a valid Sudoku.*

CHALLENGES AND SOLUTIONS

CHALLENGE: How do I convert between box-coordinates and grid coordinates? (Accessing elements within a specific box vs. accessing elements in the rows/columns of the grid)

SOLUTION: Originally planned on using two separate matrices. Figuring that was very tedious, instead I defined each box by its top-left corner and simply added on the “in-box row” or “in-box column” to the origin.